



3.4.1 Blocked Householder QR factorization

3.4.1.1 Casting computation in terms of matrix-matrix multiplication

Modern processors have very fast processors with very fast floating point units (which perform the multiply/adds that are the bread and butter of our computations), but very slow memory. Without getting into details, the reason is that modern memories are large and hence are physically far from the processor, with limited bandwidth between the two. To overcome this, smaller "cache" memories are closer to the CPU of the processor. In order to achieve high performance (efficient use of the fast processor), the strategy is to bring data into such a cache and perform a lot of computations with this data before writing a result out to memory.

Operations like a dot product of vectors or an "axpy" ($y := \alpha x + y$) perform $O(m)$ computation with $O(m)$ data and hence don't present much opportunity for reuse of data. Similarly, matrix-vector multiplication and rank-1 update operations perform $O(m^2)$ computation with $O(m^2)$ data, again limiting the opportunity for reuse. In contrast, matrix-matrix multiplication performs $O(m^3)$ computation with $O(m^2)$ data, and hence there is an opportunity to reuse data.

The goal becomes to rearrange computation so that most computation is cast in terms of matrix-matrix multiplication-like operations. Algorithms that achieve this are called *blocked algorithms*.

It is probably best to return to this enrichment after you have encountered simpler algorithms and their blocked variants later in the course, since Householder QR factorization is one of the more difficult operations to cast in terms of matrix-matrix multiplication.

3.4.1.2 Accumulating Householder transformations

Given a sequence of Householder transformations, computed as part of Householder QR factorization, these Householder transformations can be accumulated into a new transformation: If $H_0, \dots, H_{k-1}$ are Householder transformations, then

$$H_0 H_1 \dots H_{k-1} = I - UT^{-1}U^H,$$

where T is an upper triangular matrix. If U stores the Householder vectors that define $H_0, \dots, H_{k-1}$ (with "1"s explicitly on its diagonal) and t holds the scalars $\tau_0, \dots, \tau_{k-1}$, then

`T := FormT( U, t )`

computes the desired matrix T . Now, applying this UT transformation to a matrix B yields

$$(I - UT^{-1}U^H)B = B - U(T^{-1}(U^H B)),$$

which demonstrates that this operations requires the matrix-matrix multiplication $W := U^H B$, the triangular matrix-matrix multiplication $W := T^{-1}W$ and the matrix-matrix multiplication $B - UW$, each of which can attain high performance.

In [24] we call the transformation $I - UT^{-1}U^H$ that equals the accumulated Householder transformations the **UT transform** and prove that T can instead by computed as

$$T = \text{triu}(U^H U)$$

(the upper triangular part of $U^H U$) followed by either dividing the diagonal elements by two or setting them to $\tau_0, \dots, \tau_{k-1}$ (in order). In that paper, we point out similar published results [8] [36] [49] [33].

3.4.1.3 A blocked algorithm

A QR factorization that exploits the insights that yielded the UT transform can now be described:

- Partition

$$A \rightarrow \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

where A_{11} is $b \times b$.

- We can use the unblocked algorithm in Subsection 3.3.4 to factor the panel

$$\begin{pmatrix} A_{11} \\ A_{21} \end{pmatrix}$$

$$[\begin{pmatrix} A_{11} \\ A_{21} \end{pmatrix}, t_1] := \text{HouseQR_unb_var1}(\begin{pmatrix} A_{11} \\ A_{21} \end{pmatrix}),$$

overwriting the entries below the diagonal with the Householder vectors $\begin{pmatrix} U_{11} \\ U_{21} \end{pmatrix}$ (with the ones on the diagonal implicitly stored) and the upper triangular part with R_{11} .

- Form T_{11} from the Householder vectors using the procedure described earlier in this unit:

$$T_{11} := \text{FormT}(\begin{pmatrix} A_{11} \\ A_{21} \end{pmatrix})$$

- Now we need to also apply the Householder transformations to the rest of the columns:

$$\begin{aligned}
& \begin{pmatrix} A_{12} \\ A_{22} \end{pmatrix} \\
&= \\
& \left(I - \begin{pmatrix} U_{11} \\ U_{21} \end{pmatrix} T_{11}^{-1} \begin{pmatrix} U_{11} \\ U_{21} \end{pmatrix}^H \right)^H \begin{pmatrix} A_{12} \\ A_{22} \end{pmatrix} \\
&= \\
& \begin{pmatrix} A_{12} \\ A_{22} \end{pmatrix} - \begin{pmatrix} U_{11} \\ U_{21} \end{pmatrix} W_{12} \\
&= \\
& \begin{pmatrix} A_{12} - U_{11} W_{12} \\ A_{22} - U_{21} W_{12} \end{pmatrix},
\end{aligned}$$

where

$$W_{12} = T_{11}^{-H} (U_{11}^H A_{12} + U_{21}^H A_{22}).$$

This motivates the blocked algorithm in [Figure 3.4.1.1](#).

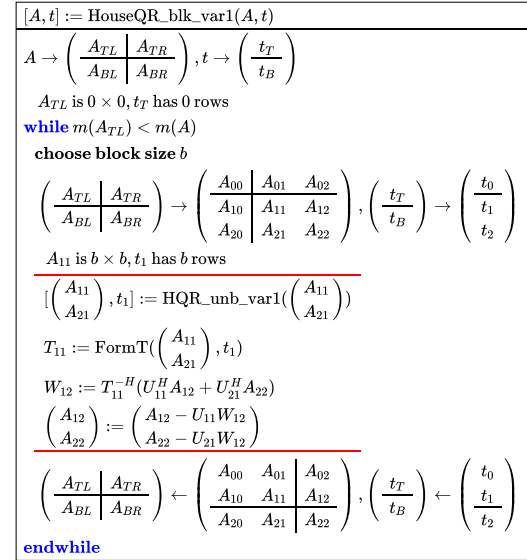


Figure 3.4.1.1. Blocked Householder transformation based QR factorization.

Details can be found in [\[24\]](#).

3.4.1.4 The WY transform ¶

An alternative (and more usual) way of expressing a Householder transform is

$$I - \beta v v^H,$$

where $\beta = 2/v^H v$ ($= 1/\tau$, where τ is as discussed before). This leads to an alternative accumulation of Householder transforms known as the compact WY transform [\[36\]](#):

$$I - USU^H$$

where upper triangular matrix S relates to the matrix T in the UT transform via $S = T^{-1}$. Obviously, T can be computed first and then inverted via the insights in the next exercise. Alternatively, inversion of matrix T can be incorporated into the algorithm that computes T (which is what is done in the implementation in LAPACK [\[1\]](#)).